Technická dokumentace k projektu: Bezpečný komunikační protokol klient-server

Předmět: BPC-AKR – Aplikovaná kryptografie (FEKT VUT)
Autoři: Gabriel Madový, Petr Věříš, Oliver Synak, Donát Szabó
Akademický rok: 2025/2026

1. Účel a popis aplikace

Cílem tohoto projektu je návrh a implementace robustního komunikačního protokolu typu klient-server, který zajišťuje důvěrnost, integritu a autentizaci přenášených dat v potenciálně nezabezpečeném síťovém prostředí. Protokol využívá kombinaci asymetrické a symetrické kryptografie k vytvoření bezpečného kanálu odolného proti odposlechu a manipulaci, včetně specifického modulu pro simulaci útoku Man-in-the-Middle (MitM).

2. Vstupy, výstupy a souborová struktura

Vstupy a výstupy

- **Vstupy:** Textové zprávy uživatele, parametry veřejných klíčů (RSA, DH) a síťové pakety ve formátech RSA_INIT, RSA_AUTH, DH_INIT a DH_ACCEPT. V případě DH se jedná konkrétně o parametry Group 14 (prvočíslo p, generátor g) a veřejný klíč protější strany. Symetrická šifra GCM vyžaduje navíc symetrický klíč (256-bit AES), inicializační vektor (nonce), zprávu k šifrování (plaintext nebo ciphertext) a autentizační tag pro dešifrování.
- **Výstupy:** Šifrované a autentizované datové pakety (ciphertext + autentizační tag), vygenerované autentizační pakety opatřené digitálním podpisem, zpětná vazba (booleovská hodnota) o validitě či kompromitaci přijatých dat a 256bitový (32bajtový) symetrický klíč pro AES.

Klíčové soubory

Soubor	Popis
`networkstack.py`	Nízkoúrovňová implementace TCP socketů s ošetřením hranic zpráv pomocí 4-bajtové hlavičky.
`RSA_Math.py`	Nízkoúrovňové kryptografické jádro pro asymetrickou autentizaci (generování prvočísel, podepisování, správa klíčů).
`RSA_Main.py`	Vyšší komunikační vrstva, orchestrace digitálních podpisů, sestavování paketů a výpočet SHA-256 hashů.
`rsa_keys.json`	Perzistence asymetrických klíčů uzlu (veřejných a privátních) pro uchování identity mezi relacemi.
`DHLogic.py`	Kryptografické jádro pro ustavení sdíleného tajemství a derivaci symetrického klíče.
`DHNetwork.py`	Orchestrace handshaku a zpracování síťových paketů pro Diffie-Hellmanovu

Soubor	Popis
	výměnu.
`GCM.py`	Autentizované symetrické šifrování AES-GCM s algoritmy GCTR a GHASH.
`main.py`	Vstupní bod s podporou módů server, client a attacker.

3. Architektura řešení

Systém je navržen jako vícevrstvý protokol:

- **Transportní vrstva:** Spolehlivý přenos bajtových proudů přes TCP.
- **Autentizační vrstva (RSA):** Identifikuje komunikující strany pomocí digitálních podpisů (SHA-256) na bázi ověřeného schématu "Hash and Sign". Využívá standardní délky klíčů (např. 1024 a 2048 bitů) pro garanci nezpochybnitelnosti.
- **Vrstva ustavení klíčů (DH):** Ephemeral Diffie-Hellman (DHE) pro unikátní symetrický klíč pro každou relaci, čímž je zajištěna vlastnost Perfect Forward Secrecy (PFS).
- **Šifrovací vrstva (AES-GCM):** Finální šifrování zpráv s podporou AEAD (Authenticated Encryption with Associated Data).

4. Použité externí knihovny

- **cryptography.hazmat:** Nízkoúrovňové operace a bezpečná implementace HKDF se SHA-256. Vyžadována pro blokovou šifru AES v módu ECB (GCTR keystream).
- **sympy:** Kryptograficky silné operace a generování náhodných, bezpečných velkých prvočísel (randprime).
- **secrets:** Generování kryptograficky silných náhodných čísel.

- **hashlib:** Výpočet hashů (SHA-256) a prevence timing útoků (`hmac.compare_digest`).
- **json, os:** Persistence asymetrických klíčů na disku.
- **cryptography.hazmat.primitives.ciphers:** Konkrétní modul pro práci s blokovými šiframi, např. AES v režimu ECB (GCTR keystream).
- **socket:** Nízkoúrovňová implementace TCP socketů pro transportní vrstvu (viz `networkstack.py`).
- **hmac:** Používáno pro kryptograficky bezpečné porovnávání autentizačních tagů (ochrana proti timing útokům).

5. Popis implementovaných algoritmů a mechanismů

5.1 Autentizace a digitální podpis (RSA)

Modul zajišťuje nepopiratelnost odesílatele a chrání integritu dat . Využívá se schéma "Hash and Sign". Pro zvýšení odolnosti je implementován padding (PSS).

Mechanismus podepisování a verifikace:

- **Odesílatel:** Prostý text (UTF-8) je redukován hashovací funkcí SHA-256 na fixní 256bitový otisk. Tento otisk je transformován na velké číslo a podepsán privátním klíčem.
- **Příjemce:** Oddělí text od podpisu, dopočítá lokálně SHA-256 hash z čistého textu a veřejným klíčem odemkne přijatý podpis. Shoda garantuje identitu a integritu.

Klíčové funkce jádra (`RSA_Math.py`):

- `generate_rsa_parameters(key_size)`: Generuje prvočísla, modul, a eulerovu funkci. Definuje fixní veřejný exponent (*e=65537*) a počítá privátní exponent.
- `rsa_sign(message_int, private_key)`: Podepisování umocněním hashe na privátní exponent.
- `rsa_verify(message_int, signature_int, public_key)`: Inverzní ověření podpisu.
- `save_keys_to_json() / load_keys_from_json()`: Persistence klíčů do JSON formátu.

Síťová orchestrace (`RSA_Main.py`):

- `build_rsa_init_packet(filename)`/ `process_rsa_init_packet(message)`: Inicializace a výměna veřejných klíčů ve formátu RSA_INIT:e:n.
- `build_rsa_auth_packet(text_message, my_private_key)`: Vytvoření zprávy s podpisem ve formátu RSA_AUTH:text_message|signature.
- `process_rsa_auth_packet(message, peer_public_key)`: Extrakce zprávy,

rekonstrukce hashe a validace podpisu.

5.2 Ustavení klíčů (Diffie-Hellman & KDF)

Výměna klíčů probíhá podle RFC 3526 (MODP Group 14) s 2048bitovým bezpečným prvočíslem a generátorem rovným 2. Veřejné klíče jsou validovány v intervalu $(1, p-1)$, aby se předešlo útokům typu Small Subgroup Attack. Surové tajemství K získané z DH výměny je pomocí mechanismu HKDF (HMAC-based Key Derivation Function) se SHA-256 extrahováno a rozšířeno na fixní délku 256 bitů (32 bajtů), což je klíč pro AES-256.

Klíčové funkce jádra (`DHLogic.py`):

- `get_dh_parameters()`: Vrací fixní parametry Group 14.
- `generate_private_key(p)`: Generuje náhodné číslo v rozsahu $[2, p-2]$.
- `is_valid_public_key(public_key, p)`: Ověřuje bezpečnost veřejného klíče.
- `generate_public_key(private_key, g, p)`: Vypočítá veřejný klíč $A = g^a \bmod p$.
- `generate_shared_secret(private_key, peer_public_key, p)`: Výpočet surového sdíleného tajemství K.
- `derive_aes_key(shared_secret)`: Transformace K na klíč pomocí HKDF-SHA256.

Síťová orchestrace (`DHNetwork.py`):

- `build_init_packet()`: Inicializuje DH výměnu na straně klienta (sestaví `DH_INIT:p:g:public_key`).
- `process_init_packet(message)`: Server rozebere `DH_INIT`, zvaliduje klíč a vrátí `DH_ACCEPT`.
- `process_accept_packet(message, p, private_key)`: Dokončení výměny na straně klienta a odvození AES klíče.

5.3 Symetrické šifrování (AES-GCM)

Využívá se Galois/Counter Mode (GCM), který kombinuje CTR mód pro šifrování (GCTR) a GHASH pro autentizaci (nad tělesem $GF(2^128)$). Zajišťuje důvěrnost i integritu dat (AEAD).

Klíčové komponenty a pomocné funkce (`GCM.py`):

- `inc32`: Inkrementuje posledních 32 bitů čítače modulo 2^{32}, což je klíčové pro generování unikátního keystreamu v GCTR pro každý datový blok.
- `xor_bytes`: Bitový XOR nad dvěma bloky, využívaný v GCTR i GHASH.
- `split_blocks`/ `split_block_no_padding`: Dělení dat na 128bitové bloky. První doplňuje bloky nulami

pro GHASH, druhý zachovává původní délku bloku.
- `gf_multiply`: Realizuje binární násobení v konečném tělese $GF(2^{128})$ pomocí bitových posunů a operace XOR, s průběžnou redukcí. Zajišťuje "míchání" dat v GHASH a je fundamentální pro garanci integrity.

Algoritmy GCTR a GHASH:
- **GCTR (Šifrování):** Pro každý blok zašifruje počítadlo pomocí AES v režimu ECB. Výsledek se vy-XORuje s plaintextem (nebo ciphertextem při dešifrování).
- **GHASH (Autentizace):** Data (AAD + ciphertext) se rozdělí na 128bitové bloky. Každý blok je XORován s předchozím stavem a vynásoben (přes `gf_multiply`) v $GF(2^{128})$. Na závěr se přidá délka dat. Výsledkem je autentizační mezivýsledek S.

Hlavní operace šifrování a dešifrování:
- `aes_gcm_encrypt(key, nonce, plaintext)`: Vytvoří hash podklíč $H=AES_K(0^{128})$ and inicializuje čítač $Y0=nonce \,||\, 1$. Zašifruje data pomocí GCTR a vypočítá $S=GHASH(H, AAD, ciphertext)$. Vygeneruje finální autentizační tag $T=GCTR(K, Y_0, S)$.
- `aes_gcm_decrypt(key, nonce, ciphertext, tag)`: Vypočítá mezivýsledek S z přijatého ciphertextu a očekávaný tag T_{calc}. Porovná přijatý tag s T_{calc} pomocí kryptograficky bezpečné funkce pro ochranu před časovými (timing) útoky (`hmac.compare_digest`). Pokud se tagy shodují, dešifruje data přes GCTR, v opačném případě vyhodí výjimku selhání autentizace.

6. Útok Man-in-the-Middle (Simulace a obrana)

Před spuštěním programu lze explicitně zvolit mód útočníka, který se navenek chová podobně jako klient, připojí se k serveru a zpřístupní uživateli CLI pro odesílání zpráv. V tomto módu je však veškerá příchozí i odchozí komunikace transparentně směrována (přeposílána) přes modul útočníka. Ten získává plný přístup ke čtení a úpravám paketů v jejich binární podobě.

Mechanismus kompromitace (MitM):
- Při iniciální výměně veřejných klíčů se útočník aktivně vloží do komunikace a podstrčí stranám svůj vlastní veřejný klíč.
- Při úspěšném podvržení identity útočník v další fázi ustavení spojení (Diffie-Hellman) vymění symetrické klíče s oběma uzly.
- Následkem toho může útočník probíhající zašifrovanou komunikaci plynule dešifrovat, odposlouchávat (či falšovat), znovu zašifrovat adekvátním symetrickým klíčem a odeslat původnímu adresátovi, aniž by došlo k výpadku TCP spojení.

Detekce a obrana:
- Útok spolehlivě selže, pokud má klient perzistentně uložený veřejný klíč serveru z předešlé, nekompromitované relace.

- Při verifikaci dojde k neshodě hashů a klient okamžitě terminálně odmítne další komunikaci se serverem.
- Pro kontext produkčního nasazení uvádíme, že v reálném prostředí se této zranitelnosti systematicky zamezuje implementací certifikátů a infrastruktury veřejných klíčů (PKI).

7. Metodika testování a výsledky

- **Funkcionální testy:** Byla potvrzena úspěšná tvorba sad klíčů, výměna paketů, generování RSA_AUTH paketů a absolutní bitová shoda derivovaných AES klíčů na straně klienta i serveru. Podpis i verifikace fungovaly bezchybně.
- **Testy integrity (MitM):** Simulována kompromitace dat na transportní vrstvě úpravou textové části. Autentizační jádro detekovalo divergenci hashů a systém útok odhalil s varováním (WARNING: Signature invalid!), spojení terminoval a ochránil příjemce. Stejně tak byl otestován GCM, který při pokusu o úpravu ciphertextu vyhodí chybu autentizace díky kontrole tagu.
- **Robustnost a test neplatných klíčů:** Potvrzena odolnost proti slabým exponentům (např. 0, 1), které interní validátor zamítl. Dále GCM modul obsahuje obranu proti časovým (timing) útokům.

8. Závěr

Protokol úspěšně kombinuje moderní kryptografii pro vysokou bezpečnost. Integrace RSA, Diffie-Hellman a AES-GCM vytváří robustní obranu vůči aktivním i pasivním síťovým hrozbám.